RecoverAI: Predictive Deleted File Recovery

CARP: Confidence-Adaptive Recovery Prioritization

Major Project External Viva Comprehensive Preparation Guide

Candidate: Asmin Shaik
Role: Research & Threat Model
Email: asminshaik2514@gmail.com
Affiliation: VIT-AP University (SCOPE)

Chapter 1: Role Overview & Executive Briefing

Role Definition: As the **Research & Threat Modeling Engineer**, my primary responsibility is establishing the physical and empirical forensic ground truth of the entire project. This involves formalizing how evidence physically decays across diverse storage architectures, designing the 24-condition experimental deletion grid, integrating the low-level raw carving engine (xCarver v4), defining multi-tier recovery strategies, and establishing cryptographic SHA-256 verification.

Spoken Pitch for External Examiners (30-Second Version):

"Good morning, respected examiners. My name is Asmin Shaik, and my individual responsibility is Research & Threat Modeling. Machine learning models in digital forensics are only as valid as the physical ground truth they train on. I formalized our forensic threat model, which captures physical cluster deallocation, ATA TRIM hardware zeroing on SSDs, and post-deletion random overwriting. I engineered our 24-condition experimental deletion grid across FAT32 and ext4 filesystems, generating 360 empirical disk traces across 15 file types. Furthermore, I integrated xCarver v4 in pure raw-signature mode using an Aho-Corasick automaton and established cryptographic SHA-256 ground truth verification across all 1,260 evaluated instances."

The 1-Minute Technical Summary:

"Digital forensic carvers historically operate under naive assumptions of contiguous data persistence. In my role, I modeled the actual operational boundaries where file recovery breaks down: first, operating system unlinking where directory tables remain in FAT32 but inode extents are immediately zeroed in ext4; second, solid-state drive background zeroing where the ATA TRIM command unmaps blocks; and third, adversarial or system slack overwriting where random data destroys cluster headers. To benchmark these threats, I designed an exhaustive combinatorial grid matrix parameterizing filesystem types, TRIM states, pre-deletion occupancy levels (10 MB vs. 50 MB), and post-deletion overwriting filler (0 MB, 5 MB, 20 MB). I coupled this with xCarver v4 running over 110+ binary signatures and authored recovery_engine/strategies.py to model computational time costs and yields across four operational carving tiers."

Chapter 2: Technical Deep Dive & Forensic Threat Formulations

1. Forensic Threat Model Dimensions:

RecoverAI evaluates evidence degradation across four core adversarial and system threats:

- **Adversarial Overwrite Contamination:** An adversary or active operating system writes random binary data into unallocated space (parameterized at 0 MB, 5 MB, and 20 MB), overwriting contiguous clusters.
- **Storage Controller TRIM Sanitization:** When files are deleted on solid-state drives, the OS issues ATA TRIM or NVMe Deallocate commands. The flash translation layer (FTL) unmaps these LBAs, causing subsequent reads to return deterministic zeros (DZAT).
- **Pre-Deletion Fragmentation:** On volumes with high occupancy (50 MB / 80% capacity), the filesystem fragments files into non-contiguous clusters, breaking the contiguous assumption of raw carving.
- **Filesystem-Specific Metadata Erasure:** In FAT32, directory entries persist with 0xE5 deletion markers until overwritten. In Linux ext4, the kernel zeroes direct and indirect inode block pointers immediately upon unlink(), making raw signature carving strictly mandatory.

2. The 24-Condition Deletion Grid Matrix (Empirical Testbed):

To evaluate these threat vectors, I formulated a combinatorial matrix spanning 2 filesystems x 2 TRIM states x 2 occupancy levels x 3 overwrite volumes = **24 distinct disk image configurations**. Running 15 standardized forensic files (PDF, JPEG, DOCX, TXT) across all 24 conditions generated 360 empirical disk traces. These were augmented with 900 forensically grounded records derived from published literature decay rules (Garfinkel 2007, Karresand 2006, Beebe 2005, Poisel 2013), yielding our official benchmark of **N = 1,260 samples**.

3. Cryptographic SHA-256 Ground Truth Verification:

Ground truth was established deterministically by hashing and byte-matching carved outputs against original pre-deletion payloads:

- ***FULL (9.61% / 121 files):*** *Cryptographic SHA-256 hash match == 100% exact bit-for-bit match.*
- ***PARTIAL (8.25% / 104 files):*** *Valid file magic header survived, and 30% <= block match ratio < 100%.*
- ***NOT_RETRIEVED (82.14% / 1,035 files):*** *Missing magic header or block match ratio < 30%. Reflects real-world storage skew where over four-fifths of deleted space is unrecoverable.*

4. Multi-Tier Strategy Costing (recovery_engine/strategies.py):

I co-designed the mathematical models parameterizing the four operational carving tiers:

- **SKIP:** Cost = 0.0 s/MB, Yield = 0.0 (Prunes unrecoverable sectors).
- **HEADER_CHECK:** Cost = 0.01 s/MB (10 ms/MB), Yield = 0.20 (Validates first 512-byte sector for magic header).

- **SCOPED_CARVE:** Cost = 0.05 s/MB (50 ms/MB), Yield = 0.70 (Scans up to expected size limit, skipping deep search).
- **FULL_CARVE:** Cost = 0.15 s/MB (150 ms/MB), Yield = 1.00 (Exhaustive sector scan until terminating EOF footer).

5. F2FS Extension (recovery_engine/f2fs_extension.py):

Based on Oh & Hwang (2025), I implemented an F2FS parser targeting the Checkpoint Area and Node Address Table (NAT) journals (F2FS_MAGIC = 0xF2F52018 at offset 0x400). F2FS was excluded from live WSL2 tests due to Microsoft kernel mount boundaries, but the parser stands fully implemented for native Linux forensics.

Parameter	Settings Evaluated	Forensic Threat Modeled
Filesystems	FAT32 (vfat), ext4	Metadata retention vs. aggressive inode zeroing
TRIM State	Enabled (True), Disabled (False)	Hardware-level SSD block unmapping and zeroing
Pre-Occupancy	Low (10 MB), High (50 MB)	Cluster fragmentation and non-contiguous cluster sprawl
Post-Overwrite	0 MB, 5 MB, 20 MB random filler	Adversarial slack overwriting and payload corruption

Chapter 3: Codebase Ownership & Key Functions

• predictor/dataset/conditions.py: Generates the 24-condition combinatorial space (all_conditions()).
• predictor/dataset/generate_deletions.py: Creates synthetic disk images (create_synthetic_disk_image()) and executes controlled file deletions.
• recovery_engine/strategies.py: Implements cost(size, level) and value(y_hat, size, level) formulas.
• recovery_engine/f2fs_extension.py: Implements F2FSVirtualAddressTable and parse_superblock().
• vendor/xcarver/carver.py: Configured with --raw-only to run pure Aho-Corasick signature carving.

Chapter 4: Cross-Module Integration (How My Work Connects)

1. **Integration with Abdul (ML & Knapsack):** My 24-condition grid and SHA-256 labels provide the training corpus for Abdul's models. My strategies.py defines the cost and yield inputs for his knapsack solver.
2. **Integration with Chandrika (Semantic Search):** My carved file payloads are ingested by Chandrika's text extractor. My block completeness metric feeds into her Fused Ranking Function.
3. **Integration with Shashwath (Database & QA):** Shashwath logs my ground-truth verdicts into the verdicts table and verifies my cost monotonicity in tests/test_orchestrator.py.

Chapter 5: Top 25 Viva Questions & Defense Answers for Asmin

Q1: What was your exact individual role in RecoverAI?

I was the Research & Threat Modeling Engineer. I designed the 24-condition deletion matrix, modeled physical data decay, integrated xCarver v4 raw carving, authored recovery strategies, and established SHA-256 ground truth verification.

Q2: Why does ext4 have so many NOT_RETRIEVED files compared to FAT32?

In ext4, the Linux kernel zeroes inode direct pointers and extents immediately upon unlinking. Because we carved in raw mode, non-contiguous fragmented files could not be reassembled without structural pointers.

Q3: What happens to a deleted file on an SSD when TRIM is enabled?

The OS issues an ATA TRIM command informing the SSD controller that LBAs are unallocated. The flash translation layer unmaps the blocks, causing subsequent reads to return deterministic zeros (DZAT).

Q4: How did you establish ground truth for your machine learning models?

We recorded original SHA-256 hashes pre-deletion. Post-carving, files matching 100% were labeled FULL; files with valid headers and 30%-99% matching blocks were labeled PARTIAL; files below 30% were labeled NOT_RETRIEVED.

Q5: What are the 4 effort tiers in your strategy module?

SKIP (cost 0, yield 0), HEADER_CHECK (cost 0.01 s/MB, yield 0.20), SCOPED_CARVE (cost 0.05 s/MB, yield 0.70), and FULL_CARVE (cost 0.15 s/MB, yield 1.00).

Q6: Why did you use xCarver v4 instead of Scalpel or Foremost?

Scalpel and Foremost recompile byte tables per chunk and lack API hooks. xCarver v4 compiles 110+ signatures into an optimized C Aho-Corasick automaton, scanning disk images in a single O(n) pass.

Q7: What does the --raw-only flag do in xCarver?

It disables filesystem-aware metadata parsing, forcing pure signature matching across raw unallocated blocks to establish an uncorrupted empirical baseline.

Q8: Why did you evaluate 24 experimental deletion conditions?

To rigorously isolate four interacting threat variables: filesystem architecture (2) x TRIM state (2) x pre-deletion occupancy (2) x post-deletion overwriting filler (3) = 24 conditions.

Q9: What were the three post-deletion overwrite volumes?

0 MB (no overwrite), 5 MB (moderate slack overwriting), and 20 MB (heavy overwriting), simulating random filler writes after file deletion.

Q10: Why was F2FS excluded from the live 24-condition testbed?

The standard WSL2 Linux kernel in our environment does not enable CONFIG_F2FS_FS by default, preventing filesystem loop mounting. The parser is fully implemented in f2fs_extension.py.

Q11: How does F2FS Virtual Address Table recovery work?

F2FS is log-structured. Node Address Table (NAT) journals in checkpoint areas preserve historical physical block addresses of unlinked files until overwritten by garbage collection.

Q12: Why did you augment the 360 empirical disk traces with 900 synthetic records?

On ext4 disks, immediate inode zeroing creates extreme class skew. To train deep learning models without severe majority overfitting, we augmented the corpus using published probabilistic decay rules.

Q13: What literature did you use to ground your synthetic data augmentation?

Garfinkel et al. (2007) for fragmented block validation, Karresand & Shahmehri (2006) for fragment distributions, Beebe & Clark (2005) for search decay, and Poisel & Tjoa (2013) for classification.

Q14: What is the magic number for F2FS?

0xF2F52018, located at byte offset 0x400 (1024 bytes) in the superblock.

Q15: What is the magic header for JPEG and PDF in your test corpus?

JPEG: 0xFFD8FFE0 (JFIF). PDF: 0x25504446 (%PDF).

Q16: What is the terminating footer for PDF and JPEG?

PDF terminates with '%%EOF'. JPEG terminates with 0xFFD9.

Q17: How does high pre-deletion occupancy affect raw carving?

When a volume is 80% full, the OS allocates files non-contiguously. Raw carving assumes cluster contiguity; hence, fragmented files suffer broken payload reassembly.

Q18: What is the difference between FS-Aware Carving and Raw Carving?

FS-aware carving reads filesystem metadata (FAT tables, NTFS MFT, ext4 inodes) to restore original names and paths. Raw carving scans binary clusters directly for magic byte signatures.

Q19: What is False Retirement Rate (FRR) in your knapsack evaluation?

FRR is the proportion of files skipped by the allocator that would have yielded recoverable data under full carving. Our FRR was 15.18%.

Q20: Why did you cost SCOPED_CARVE at 0.05 s/MB vs. FULL_CARVE at 0.15 s/MB?

SCOPED_CARVE stops reading blocks once predicted file size is reached, saving 66% of I/O time compared to FULL_CARVE which scans until EOF footers or cluster timeouts.

Q21: How does your module feed into Chandrika's semantic search?

My verification logic calculates BlockCompleteness for carved files, which Chandrika uses as a 15% weight in her Fused Ranking Function.

Q22: How does your module feed into Abdul's machine learning model?

My 24-condition grid and SHA-256 ground truth labels form the exact training corpus (N = 1,260) used to train RecoverabilityNet and XGBoost.

Q23: How does your module feed into Shashwath's database layer?

My ground truth hashes and carved byte lengths are logged into Shashwath's verdicts table via SQLAlchemy ORM.

Q24: What baseline recovery accuracy did Yermolov (2025) report?

Yermolov reported a baseline recovery accuracy of approximately 0.80 across enterprise platforms, which our ensemble exceeds at 0.8270.

Q25: What future improvement would you make to xCarver?

Implement bi-directional LSTM or Transformer byte-continuity classifiers inside xCarver to stitch non-contiguous clusters across fragmented ext4 images.

Chapter 6: Rapid Revision Cheat Sheet for Asmin

- ***Grid Matrix:*** *2 filesystems (FAT32, ext4) x 2 TRIM states x 2 occupancies (10MB, 50MB) x 3 overwrites (0, 5, 20MB) = 24 conditions*
- ***Dataset:*** *N = 1,260 (360 real empirical + 900 synthetic)* | ***Verdicts:*** *1,035 NOT_RETRIEVED, 104 PARTIAL, 121 FULL*
- ***Effort Tiers:*** *SKIP (0s, 0.0y), HEADER_CHECK (0.01s, 0.20y), SCOPED (0.05s, 0.70y), FULL (0.15s, 1.00y)*
- ***Carver Engine:*** *xCarver v4 with C-based Aho-Corasick automaton across 110+ signatures in --raw-only mode*
- ***Ground Truth:*** *SHA-256 exact match (FULL), 30%-99% block match (PARTIAL), <30% match (NOT_RETRIEVED)*
- ***Key Code:*** *conditions.py, generate_deletions.py, recovery_engine/strategies.py, f2fs_extension.py*